Aula 02 – Introdução aos Sistemas Distribuídos

1. Definição

Um sistema distribuído é uma coleção de elementos de computação autônomos que os usuários veem como um sistema único e coerente.

- **Nós autônomos:** dispositivos de hardware ou processos de software, cada um com sua própria percepção de tempo.
- **Sistema coerente:** o usuário vê um só sistema, então os nós precisam colaborar.
- **Consequência central:** não existe relógio global. Isso causa problemas de sincronização e coordenação.
- **Associação ao grupo:** é preciso gerenciar quem participa e saber se você fala com um membro autorizado.

Analogia: uma orquestra sem maestro e sem relógio na parede. Cada músico tem seu próprio tempo, mas o público precisa ouvir uma só música.

2. Organização

Redes de overlay

Cada nó se comunica só com seus vizinhos. O conjunto de vizinhos pode mudar. A overlay não precisa seguir a estrutura física da rede.

Tipo	Como funciona	Exemplo
Estruturada	Cada nó tem vizinhos bem definidos	Árvore, anel
Não estruturada	Cada nó conhece nós escolhidos aleatoriamente	Peer-to-peer (P2P)

Em P2P, a falha de um peer não compromete o sistema.

Middleware

É o "SO dos sistemas distribuídos". Fica entre os SOs locais e as aplicações. Reúne funções que as aplicações não precisam reimplementar, como replicação, segurança e broadcast.

3. Objetivos

3.1 Compartilhamento de recursos

3.2 Transparência de distribuição

Transparência é esconder um aspecto da distribuição.

Tipo	O que esconde
Acesso	Diferenças de representação de dados e de invocação
Localização	Onde o objeto está
Relocalização	Que o objeto pode mudar de lugar enquanto está em uso
Migração	Que o objeto pode ser movido
Replicação	Que o objeto tem cópias
Concorrência	Que o objeto é compartilhado entre usuários independentes
Falhas	Falhas e a recuperação do objeto

3.3 Abertura

Um sistema aberto interage com outros sistemas abertos:

- Respeita interfaces bem definidas: atende.
- Permite portabilidade de aplicações: atende.
- É fácil de estender: atende.
- É facilmente interoperável: não atende.

3.4 Escalabilidade

Dimensão	Mede	Problema principal
Tamanho	Nº de usuários/processos	Limites de CPU, disco e largura de banda
Geográfica	Distância entre nós	LAN para WAN: mais latência, links pouco confiáveis, sem broadcast de rede
Administrativa	Nº de domínios administrativos	Políticas conflitantes de uso, pagamento, gerenciamento e segurança (ex.: LGPD)

Sistema descentralizado

- Nenhuma máquina tem a informação completa do estado do sistema.
- As máquinas decidem com base em informação local.
- A falha de uma máquina não deve arruinar o algoritmo.
- Não se pode assumir um relógio global.

Estimativa back-of-the-envelope

Dados: 400 milhões de usuários ativos, 50% usam por dia, 2 tweets por usuário/dia, 10% dos tweets com 1 MB de mídia, retenção de 5 anos.

- Usuários por dia: 400 mi × 0,5 = 200 mi
- TPS: (200 mi × 2) / (24 × 3600) ≈ 4.630 tweets/s
- Pico (2×): ≈ 9.260 tweets/s
- Mídia por dia: 200 mi × 2 × 0,1 × 1 MB ≈ 38 TB
- Mídia em 5 anos: ≈ 68 PB

Técnicas de escalabilidade

Técnica	Ideia	Exemplos
Esconder latência	Não espere a resposta; faça outra coisa	Comunicação assíncrona, handlers multithread
Particionamento	Divida dados e computação entre máquinas	DNS, WWW, microsserviços, sharding
Replicação e cache	Faça cópias em várias máquinas	Sites espelhados, cache de navegador, memcached

O problema da escalabilidade

1. Várias cópias geram inconsistência: alterar uma cópia a torna diferente das outras.
2. Manter as cópias consistentes exige sincronização global a cada modificação.
3. Não há relógio global, e a sincronização global impede soluções escaláveis.

Saída: tolerar algum grau de inconsistência. Isso depende da aplicação.

4. Modelos de sistema

Modelo	Limites de tempo (mensagem e ação)	Exemplo
Síncrono	Conhecidos	LAN, datacenters
Assíncrono	Nenhum	Internet
Parcialmente síncrono	Começa assíncrono e eventualmente vira síncrono	Internet

5. Armadilhas - hipóteses falsas:

1. A rede é confiável.
2. A rede é segura.
3. A rede é homogênea.
4. A topologia não muda.
5. A latência é zero.
6. A largura de banda é infinita.
7. O custo de transporte é zero.
8. A rede tem um administrador.

Aula 03 – Tipos de Sistemas Distribuídos

1. Visão geral

- Sistemas para computação distribuída de alto desempenho: cluster, grade e nuvem.
- Sistemas de informação distribuídos.
- Sistemas distribuídos para computação pervasiva.

2. Computação de alto desempenho

Tipo	Características
Cluster	Computadores de boa qualidade ligados por LAN. Homogêneo (mesmo SO, hardware quase idêntico). Um único nó gerenciador (master) e vários nós de computação.
Grade	Nós heterogêneos, espalhados entre organizações ou pessoas, em geral em rede de longa distância. Usa organizações virtuais: IDs de usuários agrupados com autorização para usar certos recursos.
Nuvem	Recursos oferecidos em quatro camadas (abaixo).

Camadas da nuvem

Camada	O que cobre	Exemplo
Hardware	Processadores, roteadores, energia, refrigeração (escondido do cliente)	Datacenters
Infraestrutura (IaaS)	Virtualização para alocar servidores e armazenamento virtuais	Amazon EC2
Plataforma (PaaS)	Abstrações de alto nível para serviços	MS Azure, Amazon S3
Aplicação (SaaS)	As aplicações em si	Google Apps, YouTube

3. Sistemas de informação distribuídos

As palavras-chave são **transação** e **integração**. Muitas aplicações dessa classe se conectam à rede para trocar informações.

Transações e ACID

Uma transação é uma operação atômica. Se algo falhar, ela é abortada (ABORT_TRANSACTION). O exemplo do slide atualiza dois bancos independentes (companhia aérea e hotel) como subtransações aninhadas.

Propriedade	Significado
Atomicidade	Todas as operações têm sucesso ou todas falham. Em falha, o estado fica inalterado.
Consistência	A transação leva a um estado válido. Estados intermediários inválidos podem existir durante a execução.
Isolamento	Transações concorrentes não interferem. Para uma transação T, as outras ocorrem antes ou depois de T.
Durabilidade	Depois do término, os efeitos são permanentes e sobrevivem a falhas.

TP Monitor: quando os dados de uma transação estão em vários servidores, o monitor coordena a execução.

Integração de aplicações (EAI)

Um TP Monitor não basta. Aplicações precisam se comunicar via middleware:

- **RPC:** o cliente chama o método como se fosse local, mas ele pode rodar em outra máquina. Exemplos: Java RMI e gRPC. Um **proxy** encaminha as requisições ao servidor responsável.
- **MOM** (middleware orientado a mensagens).

4. Sistemas pervasivos

Tendência em que os nós são pequenos, móveis e embutidos em sistemas maiores. Ligados à Internet das Coisas (IoT). Os três tipos se sobrepõem.

Tipo	Pontos principais
Ubíquos	Sempre presentes. Requisitos: mudança contextual, autonomia (sem intervenção humana) e inteligência (comportamento emergente, como o flocking).
Móveis	Smartphones, tablets, óculos AR/VR. Comunicação sem fio. A localização muda, então a palavra-chave é descoberta. Rotas instáveis levam às redes tolerantes a disrupções. Estudos de rastreamento de celulares mostram que as pessoas tendem a voltar aos mesmos lugares.
Sensores	Muitos nós (dezenas a milhares), simples (pouca memória, computação e comunicação) e em geral com bateria. Sentem e atuam no ambiente.

Rede de sensores como sistema distribuído: (a) os dados vão direto ao operador, ou (b) cada sensor processa e armazena dados e envia só respostas a consultas.

Visão final do slide: Cloud, Edge e Things.

Aula 04 – Arquiteturas de Sistemas Distribuídos

1. Estilos arquiteturais

Um estilo é um conjunto de componentes interligados que interagem entre si.

Estilo	Ideia	Exemplo
Camadas	Componentes logicamente diferentes, distribuídos entre máquinas. O fluxo de requisição desce e o de resposta sobe.	Cliente-servidor; buscador com camadas de interface, processamento e dados
Objetos	Objetos distribuídos. Chamadas parecem locais, mas podem ser a objetos remotos.	Java RMI, microsserviços
Eventos e dados	Desacoplam os processos. Publish/subscribe: desacoplado na referência (anônimos). Espaço de dados compartilhados: desacoplado na referência e no tempo.	Barramento de eventos; Linda/Rinda
Recursos	Coleção de recursos gerenciados por componentes (REST). Identificação por nome único, mesma interface em todos os serviços, serviço sem memória do chamador.	Amazon S3

REST: operações básicas

Operação	Descrição
PUT	Cria um novo recurso
GET	Recupera o estado de um recurso
DELETE	Apaga um recurso
POST	Modifica um recurso ao transferir um novo estado

No S3, objetos ficam em buckets, com URI do tipo http://BucketName.s3.amazonaws.com/ObjectName. Criar usa PUT, listar usa GET no bucket e ler usa GET na URI completa.

2. Arquiteturas de software

Centralizadas e multicamadas

- **Cliente-servidor** (desde os anos 70): servidores oferecem serviços, clientes os usam, em máquinas possivelmente diferentes, com requisição e resposta. Um processo pode ser cliente e servidor ao mesmo tempo.
- **Três camadas lógicas:** apresentação (interface), negócio (aplicação) e dados (banco).
- **Camadas físicas:** uma (terminal burro e mainframe), duas (cliente e servidor único) ou três (cada camada em uma máquina).

Descentralizadas (P2P)

Tipo	Como funciona
Estruturado	Nós organizados em uma estrutura distribuída específica (anel, árvore, tabela de hash). Cada peer é cliente e servidor. Cada nó fica responsável por chaves, e a operação LOOKUP(key) roteia até o nó correto.
Não estruturado	Cada nó mantém uma visão parcial com c vizinhos e troca informações periodicamente (gossip). Busca por flooding e random walk. Superpeers podem manter índices, monitorar a rede ou configurar conexões (ex.: Skype).
Híbrido	Alguns nós têm funções especiais, ou mistura de cliente-servidor com P2P.

Quase todos são redes overlay: os dados seguem conexões definidas pelos nós.

Híbridas

- **Edge e CDN:** servidores de borda levam o conteúdo para perto do cliente.
- **BitTorrent:** o cliente obtém um arquivo .torrent, consulta o tracker (lista de nós que guardam o arquivo) e entra em um swarm. Ali recebe pedaços em paralelo e os redistribui.

3. Arquitetura versus middleware

O estilo escolhido pode não ser o melhor em todos os casos. O middleware permite adaptar o comportamento dinamicamente.

- **Interceptors:** interceptam o fluxo normal quando um objeto remoto é invocado, no nível da requisição ou da mensagem.
- **Wrappers e brokers: conectam aplicações diferentes para acessar suas funcionalidades. Wrappers adapta a interface de uma aplicação. Broker intermedia a comunicação entre várias aplicações.**

Aula 05 – Processos, Threads e Migração

1. Processos e threads

- **Processo:** abstração do SO que representa a execução de um programa. Tem memória isolada por segurança (evita ataques e bugs).
- **Thread:** linha de execução dentro de um processo. Todas as threads compartilham memória e recursos do processo.
- **Troca de contexto:** entre threads é barata (troca o IP e alguns registradores). Entre processos é cara (tabela de páginas, rotinas de interrupção).
- Processos ainda podem ser preferíveis em alguns casos. Exemplo: Google Chrome.

No Linux: fork (C) e os.fork (Python) criam processos. pthread (C) e Thread (Python e Java) criam threads.

Concorrência

Threads acessam as mesmas estruturas ao mesmo tempo. Sem controle ocorre **condição de corrida**. Exemplo do slide: id++ tem leitura, incremento e escrita, e duas threads podem ler o mesmo valor. A solução é sincronizar (lock, semáforo, monitor), cuidando para evitar deadlocks.

2. Threads em sistemas distribuídos

- **Evita bloqueios:** com uma só thread, uma chamada de E/S bloqueia tudo.
- **Explora paralelismo:** várias threads rodam em vários cores.
- **Evita troca de contexto entre processos:** memória compartilhada.

No cliente: o navegador baixa cada arquivo da página em uma thread própria e esconde a latência da rede. Um cliente também pode fazer várias chamadas remotas ao mesmo tempo. Com servidores diferentes, o ganho pode ser linear.

No servidor: iniciar uma thread é bem mais barato que um processo, e servidores de uma só thread não usam multi-core. O modelo clássico é **dispatcher/worker**: uma thread recebe a requisição da rede e a repassa a uma thread operária. Estruturas compartilhadas continuam sujeitas à condição de corrida.

3. Clientes

O cliente fala com o servidor por protocolo de aplicação ou de middleware. O software cliente pode esconder a distribuição:

- **Acesso:** stubs do cliente para RPC.
- **Localização e migração:** o cliente acompanha a localização atual.
- **Replicação:** o stub do cliente gerencia as requisições replicadas a vários servidores.
- **Falhas:** muitas vezes só o cliente tenta esconder falhas de comunicação e do servidor.

4. Servidores

Tipo	Descrição
Iterativo	Trata uma requisição antes de aceitar a próxima
Concorrente	Um dispatcher repassa cada requisição a uma thread ou processo. É o mais comum, principalmente com operações bloqueantes (disco, outros servidores).

Stateless e stateful

	Stateless	Stateful
Estado do cliente	Não guarda após processar a requisição	Guarda (ex.: arquivo aberto, cache do cliente)
Vantagens	Cliente e servidor independentes; menos	Pode ter desempenho muito alto, com prefetching e cópias

	Stateless	Stateful
	inconsistência por falhas	locais
Desvantagens	Possível perda de desempenho; sem antecipar o cliente	O problema principal é a inconsistência, não a confiabilidade
Exemplo do slide	Página inicial sem login nem cookies	Home após o login

Aglomerados de servidores (três camadas)

A primeira camada (frontend) recebe os clientes e repassa as requisições ao backend (lógica e computação), que usa a terceira camada (arquivos e bancos distribuídos). O **logical switch** pode ser um proxy conhecido como **load balancer**, que escolhe o servidor por estratégias como round-robin, ociosidade ou sessão. Como o frontend sobrecarrega fácil, usam-se switch de camada de transporte (só redireciona) ou distribuição por conteúdo da mensagem.

5. Migração de código

Motivos: distribuir carga, reduzir comunicação (código perto dos dados) e dar flexibilidade (mover código ao cliente quando necessário).

Modelo	O que se move
CS (cliente-servidor)	Nada. O código e os recursos ficam no servidor.
REV (avaliação remota)	O código vai do cliente para o servidor e é executado lá.
CoD (código sob demanda)	O código vai do servidor para o cliente.
MA (agentes móveis)	Código e estado de execução se movem entre máquinas.

Sistemas heterogêneos: a máquina de destino pode não rodar o código migrado, pois o contexto de execução depende de hardware, SO e bibliotecas. A solução é uma máquina abstrata: linguagens interpretadas com sua MV, uma MV que representa um SO completo (VirtualBox, 2007) ou um container (Docker, 2013).

Casos citados: arquitetura resiliente na AWS e a arquitetura da Wikimedia (caches de texto e de upload, máquinas de processamento e de redimensionamento, armazenamento tipo S3).

Aula 06 – Comunicação

1. Fundamentos

Camadas: baixo nível, transporte, middleware e aplicação. A camada de transporte (TCP e UDP) é a mais usada pelos sistemas distribuídos. TCP é orientado a conexão, confiável e a fluxo. UDP é não confiável (best-effort). IP multicast não deve ser tratado como serviço padrão.

Middleware: oferece protocolos de comunicação, (des)empacotamento (marshaling), nomes, segurança e escalabilidade (replicação, sharding, cache). Sobram os protocolos específicos da aplicação.

Eixo	Opção A	Opção B
Persistência	Transiente: o remetente descarta a mensagem se ela não puder ser entregue	Persistente: a mensagem é guardada até a entrega
Sincronia	Síncrona: o remetente espera	Assíncrona: o remetente continua

Pontos de sincronização possíveis: no envio da requisição, na entrega da requisição ou após o processamento.

2. RPC (chamada a procedimento remoto)

Esconde a comunicação atrás de uma chamada de procedimento (transparência de acesso). É precursor dos microsserviços.

Passos de uma chamada

1. O cliente chama o stub do cliente.
2. O stub empacota os parâmetros e chama o SO local.
3. O SO envia a mensagem ao SO remoto, que a entrega ao stub do servidor.
4. O stub do servidor desempacota e faz a chamada local.
5. O resultado volta pelo mesmo caminho até o cliente.

Concorrência: normalmente usa dispatcher/worker, com uma thread por requisição. Variáveis e estruturas precisam de proteção.

Variante	Comportamento
Síncrona	O cliente espera o resultado
Assíncrona (sem resposta)	O cliente continua após o servidor aceitar a requisição (one-way)
Síncrona diferida	O cliente continua e o servidor devolve o resultado depois (callback). O cliente também pode fazer poll.
Para um grupo	A mesma requisição vai a vários servidores

RMI (Java): uma interface estende Remote, a classe estende UnicastRemoteObject e implementa a interface. O servidor cria o registry (createRegistry) e faz bind. O cliente faz lookup.

gRPC (Google, 2015): segundo o slide, mensagens de 3 a 10 vezes menores e envio de 20 a 100 vezes mais rápido que alternativas como REST. Útil para baixa latência, alta escalabilidade e clientes móveis. A interface é definida em arquivo de serviço e mensagens.

3. Comunicação orientada a mensagens (MOM)

Comunicação assíncrona e persistente com filas gerenciadas pelo middleware.

Operação	Efeito
PUT	Adiciona uma mensagem à fila
GET	Bloqueia até haver mensagem e remove a primeira
POLL	Verifica se há mensagem, sem bloquear
NOTIFY	Instala um tratador chamado a cada nova mensagem

Message broker: componente centralizado que trata a heterogeneidade (converte formatos) e gerencia as filas.

Apache Kafka (LinkedIn, 2012)

- Usa um **event log** (append-only), chamado **tópico**. Cada registro tem um identificador (offset) que captura a ordem.
- O produtor envia eventos de forma assíncrona. Os eventos ficam persistidos e vários consumidores os leem. Cada consumidor indica o log e o offset.
- O broker é stateless em relação às requisições dos clientes.
- **Escalabilidade:** particionamento (sharding) do tópico. O produtor escolhe a partição, em geral pelo hash de uma chave.
- **Disponibilidade:** replicação. Um líder atende leituras e escritas, e N seguidores dão resiliência.

Números do slide: Slack (2018) com mais de 1 bilhão de mensagens por dia e 16 brokers. PayPal (2022) com 1,3 trilhão de mensagens por dia e 1.500 brokers.

Apache ActiveMQ: modelo ponto a ponto (só um consumidor recebe a mensagem da fila) e modelo publish/subscribe (todos os consumidores recebem).

4. Multicast

No unicast há um destino. No multicast, o emissor envia a mesma mensagem a um grupo. Pode ser feito na rede (difícil) ou no middleware. No middleware, **receber** vem da rede e **entregar** é decisão do middleware.

Ordem	Regra
FIFO	Se um processo p envia m1 e depois m2, todos entregam m1 antes de m2
Causal	Se m1 precede causalmente m2, todos entregam m1 antes de m2, não importa quem enviou
Total	Se um processo entrega m1 antes de m2, todos entregam nessa mesma ordem

5. Gossip e flooding

Um nó envia a mensagem a vizinhos (gossip) e cada vizinho a reenvia se ainda não a viu (flooding). Mais conexões significam mais custo.

Anti-entropy

Cada nó escolhe outro aleatoriamente e troca estado. Variantes: **pull** (P só pede), **push** (P só envia) e **push-pull** (ambos). O push-pull espalha a atualização em O(log N) rodadas.

Rumor

Um nó com atualização contata outros. Se o contatado já a tem, o nó para com probabilidade p_{stop}. A fração s de nós ignorantes segue s = e^(-(1/p_stop + 1)(1 - s)). Com 10.000 nós: 1/p_stop = 1 deixa cerca de 2032 nós sem a atualização, e 1/p_stop = 7 deixa cerca de 3.

Para garantir que todos os nós sejam atualizados, só o rumor não basta.

Aula 09 (parte 1) – Nomes

1. Definições

Nomes denotam **entidades**. Para operar sobre uma entidade, é preciso um **ponto de acesso**, que é identificado por um **endereço**.

- **Nome puro (plano):** sem significado próprio, só uma string aleatória. Serve apenas para comparação.
- **Identificador:** um nome com três propriedades: (1) refere-se a no máximo uma entidade; (2) cada entidade tem no máximo um identificador; (3) sempre se refere à mesma entidade, sem reutilização.
- Um identificador não precisa ser um nome puro. Pode ter conteúdo.

2. Espaço de nomes plano

Problema: dado um nome não estruturado, como achar seu ponto de acesso? Os slides listam quatro abordagens: (1) soluções simples (broadcast, flooding, ponteiros), (2) baseadas em local pré-determinado (home-based), (3) DHT e (4) serviço hierárquico.

Broadcasting

Envia o ID a todos e pede que a entidade devolva o endereço. Não escala além de redes locais e obriga todos os processos a ouvir e processar os pedidos. Exemplo: **ARP**, que pergunta "quem tem este IP?" para achar o endereço MAC.

Flooding

O nó X envia a requisição R por um recurso A a alguns vizinhos V. Cada V:

1. ignora se já viu a requisição;

2. procura A localmente;
3. se não tem A, encaminha R a vizinhos;
4. se tem A, responde direto a X ou a quem encaminhou.

Tem problema de escalabilidade de tamanho e geográfica. O **TTL** limita o alcance: só encaminha se TTL > 0, e passa TTL-1 adiante.

Gnutella: P2P de compartilhamento de arquivos. Busca por flooding com TTL: encaminha a 7 vizinhos com TTL=10. A resposta volta pelo caminho inverso. As transferências são diretas entre nós, via HTTP.

DHT Chord (visão rápida)

Nós em um anel lógico com IDs aleatórios de m bits. Cada entidade tem uma chave de m bits e fica sob o nó de menor id ≥ k (seu sucessor). Busca linear pelo anel é ruim, então cada nó p mantém uma **finger table** com até m entradas: FT_p[i] = succ(p + 2^(i-1)), ou seja, atalhos a distâncias 1, 2, 4, 8... Para a chave k, p encaminha ao nó FT_p[j] tal que FT_p[j] ≤ k < FT_p[j+1]. Veja a aula 09 (Chord) para os detalhes.

Serviço hierárquico de localização (HLS)

Árvore em larga escala: a rede é dividida em domínios hierárquicos, e cada domínio tem um nó de diretório. O endereço de uma entidade E fica em uma folha ou em um nó intermediário.

- **Invariantes:** um nó intermediário tem ponteiro para um filho se e somente se a subárvore desse filho contém o endereço de E. A raiz conhece todas as entidades.
- **Consulta:** começa na folha local. Se o nó conhece E, desce pelo ponteiro. Se não, sobe, até chegar à raiz se necessário.
- **Inserção:** o pedido sobe até o primeiro nó que conhece E. Cria-se uma cadeia de ponteiros até a folha, que guarda o endereço.

3. DNS

Resolução de nomes é o mecanismo que recupera a entidade que tem o nome. O desafio é distribuir a resolução e o gerenciamento do espaço de nomes por várias máquinas, com escalabilidade e disponibilidade. O DNS (RFC de 1983, implementado em 1984) traduz um nome em IP.

Três níveis

Item	Global	Administração	Gerencial
Escala geográfica	Mundial	Organização	Departamento
Nº de nós	Poucos	Muitos	Quantidades enormes
Responsividade	Segundos	Milissegundos	Imediato
Propagação de atualizações	Lenta	Imediata	Imediata
Nº de réplicas	Muitos	Nenhum ou poucos	Nenhum
Cache no cliente	Sim	Sim	Às vezes

O nível global tem os diretórios de topo (com, edu, org, jp...), gerenciados em conjunto e muito estáveis. O de administração agrupa diretórios de nível intermediário, sob administradores diferentes. O gerencial tem nós de um único administrador, com mudanças regulares.

Resolução iterativa e recursiva

	Iterativa	Recursiva
Quem percorre os servidores	O cliente. Cada servidor devolve o endereço do próximo.	Cada servidor repassa ao próximo e espera o resultado.
Exemplo	O cliente pergunta ao root, depois ao servidor nl, depois ao vu, e assim por diante.	O cliente pergunta só ao root, que encadeia as consultas e devolve a resposta final.

Na resolução recursiva com cache, os servidores guardam resultados intermediários (por exemplo, #<vu> e #<vu,cs>). Isso acelera, mas o uso extensivo de cache pode gerar inconsistências.

Escalabilidade

- **Tamanho:** servidores de alto nível recebem muitas requisições. Como os nós global e de administração mudam pouco, usa-se replicação extensiva e inicia-se a resolução no servidor mais próximo.
- **Geográfica:** a resolução recursiva reduz comunicação de longa distância em comparação à iterativa. Mapear nós a servidores em qualquer lugar cria uma dependência de localização implícita.

Registros de um nó DNS

Tipo	Descrição
SOA	Informações sobre a zona
A	Endereço IP do host que o nó representa
MX	Servidor de e-mail do nó
SRV	Servidor de um serviço específico
NS	Servidor de nomes da zona
CNAME	Link simbólico (alias)
PTR	Nome canônico de um host
HINFO	Informações sobre o host
TXT	Qualquer informação útil

O slide mostra um dig ufabc.edu.br com registros SOA, NS, A, MX e TXT, além de endereços A e AAAA dos servidores de nomes.

Aula 09 (parte 2) – DHT Chord

1. Tabela de hash distribuída (DHT)

- Sistema P2P estruturado: cada peer é cliente e servidor.
- Criada em 2001 para achar um recurso com eficiência, em O(log N).
- Dá escalabilidade de tamanho e particionamento (sharding).
- Evita o ponto único de falha (SPoF).

Ideia: uma tabela de hash em um só servidor vira um SPoF se ele cair. A solução é distribuir a tabela por vários nós.

2. Construção do Chord

Três decisões: (1) definir um espaço comum de valores para nós e itens; (2) conectar os nós com poucos links, para minimizar saltos; (3) definir como designar itens aos nós.

- **Espaço de IDs:** {0, 1, ..., 2^M-1}, formando um anel módulo 2^M.
- **Nós:** cada nó recebe um número obtido por uma função de hash H.
- **Responsabilidade:** cada nó cuida de um conjunto disjunto de IDs, de (ID do nó anterior + 1) até o seu. No exemplo com M=4 e nós 0, 2, 5, 6, 11: o nó 0 cuida de [12, 0], o nó 5 de [3, 5] e o nó 11 de [7, 11]. É o particionamento.
- **Sucessor:** succ(x) é o primeiro nó, no sentido horário, com id ≥ x. Exemplo: succ(12) = 0, succ(1) = 2, succ(6) = 6.
- **Ligação:** cada nó p aponta para succ(p+1). Invariante: um nó sempre tem conexão com seu sucessor.

3. Armazenar itens

Cada item <key, value> recebe o identificador k = H(key), com a mesma função H, e é guardado no **sucessor** de k. Exemplo dos slides: H('Seif') = 9, então o item fica no nó 11. H('Fatemeh') = 12 e H('Sarunas') = 14 vão para o nó 0. H('Cosmin') = 2 fica no nó 2. H('Tallat') = 4 vai para o nó 5.

Consistent hashing: técnica em que só O(m/N) pares chave-valor são transferidos quando nós entram ou saem (m chaves, N nós). O slide pergunta qual seria o problema de guardar o item no nó H(key) % N.

4. Busca

Para buscar a chave k, pergunta-se a qualquer nó, calcula-se H(k) e seguem-se os ponteiros de sucessor até achar. Exemplo: get('Seif') a partir do nó 2 passa pelos nós 2, 5, 6 e 11 e devolve "Stockholm". Só com sucessores, a busca é O(N).

Algoritmo findSuccessor

- Se o id está em (predecessor, n], o nó n é a resposta.
- Se o id está em (n, sucessor], o sucessor é a resposta.
- Senão, encaminha a pergunta pelo anel (versão simples) ou ao nó mais próximo anterior ao id (com finger table).

put e get usam findSuccessor. Quando a resposta chega, o iniciador pede ao nó achado que guarde (store) ou devolva (retrieve) o valor. Notação: (a, b] exclui a e inclui b.

5. Buscas rápidas com finger table

Cada nó p aponta para succ(p+1), succ(p+2), succ(p+4), ..., succ(p+2^{M-1}). Assim, FT_p[i] = succ(p + 2^(i-1)), para i = 1..M. A tabela tem M entradas, e o espaço guarda 2^M chaves. Qualquer nó alcança outro em cerca de $\log_2(N)$ saltos. Exemplo do slide: $\log_2(1.000.000)$ é cerca de 20. Em implementações, normalmente M = 128.

closestPrecedingNode(id): percorre a finger table de i = M até 1 e devolve a primeira entrada que está no intervalo (n, id). Se nenhuma estiver, devolve o próprio n.

Exemplo: finger table do nó 0 (M=4, nós 0, 2, 5, 6, 11)

i	n + 2^(i-1)	succ
1	0 + 1 = 1	2
2	0 + 2 = 2	2
3	0 + 4 = 4	5
4	0 + 8 = 8	11

Exemplo: get(15) a partir do nó 1

Conforme as tabelas dos slides (M=4):

Nó	Finger table	Próximo salto
1	[3, 3, 6, 9]	Nó 9: o mais distante que ainda precede 15
9	[10, 14, 14, 1]	Nó 14
14	[15, 0, 3, 6]	15 está em (14, sucessor=15], então o sucessor 15 responde

Foram 3 saltos, em vez de percorrer todos os nós pelo anel.